

MennaTallah Ahmed	2023100
Nour Mohammed	20236110
Shahd Osama	20236052
Aya Moshen	20236021
Jana Ayman	20236025
Mohammed Khaled	20236084

Section Number: S1

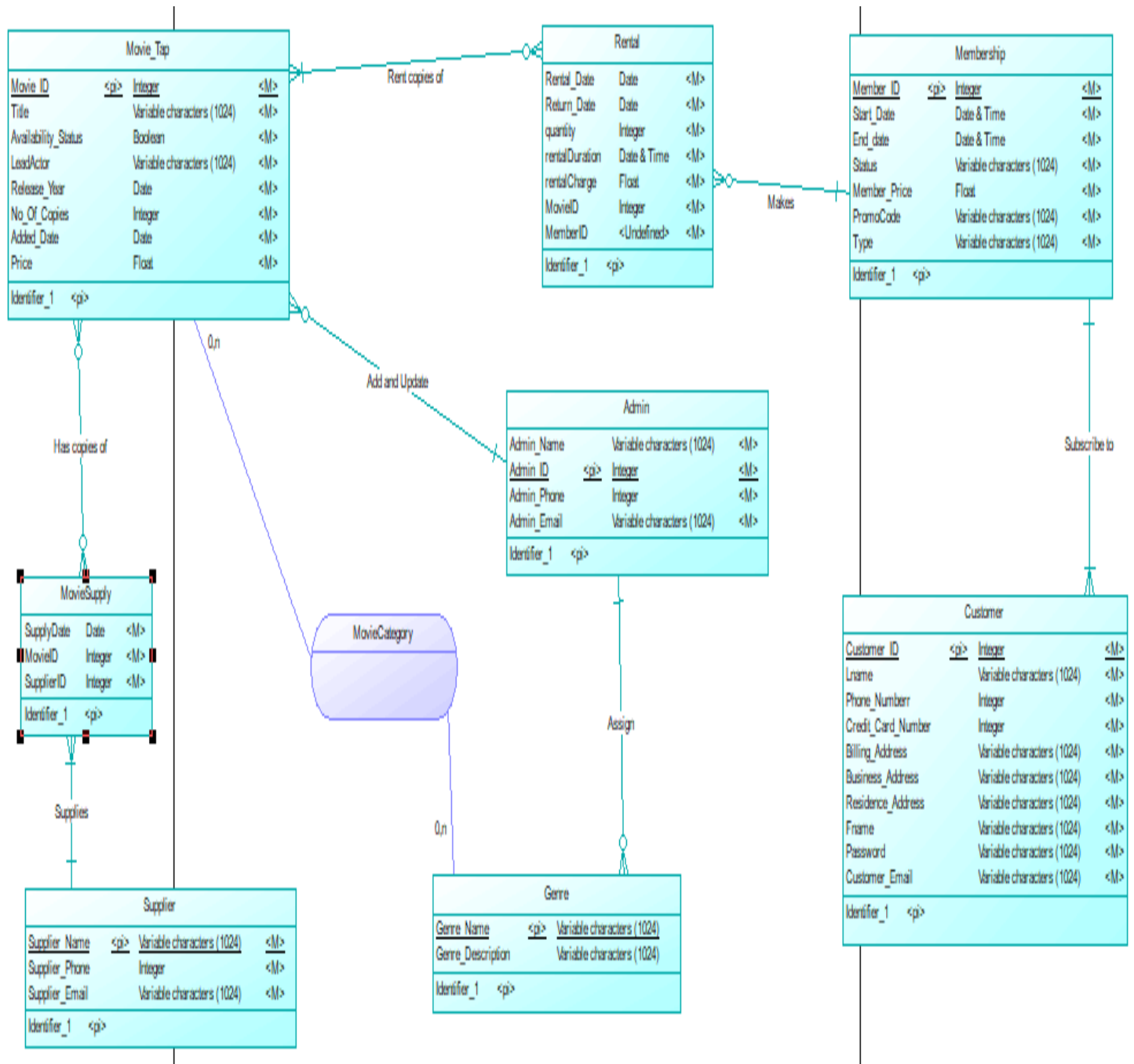
TA:Ibrahim Gomaa

Project:Movie Rental Management System

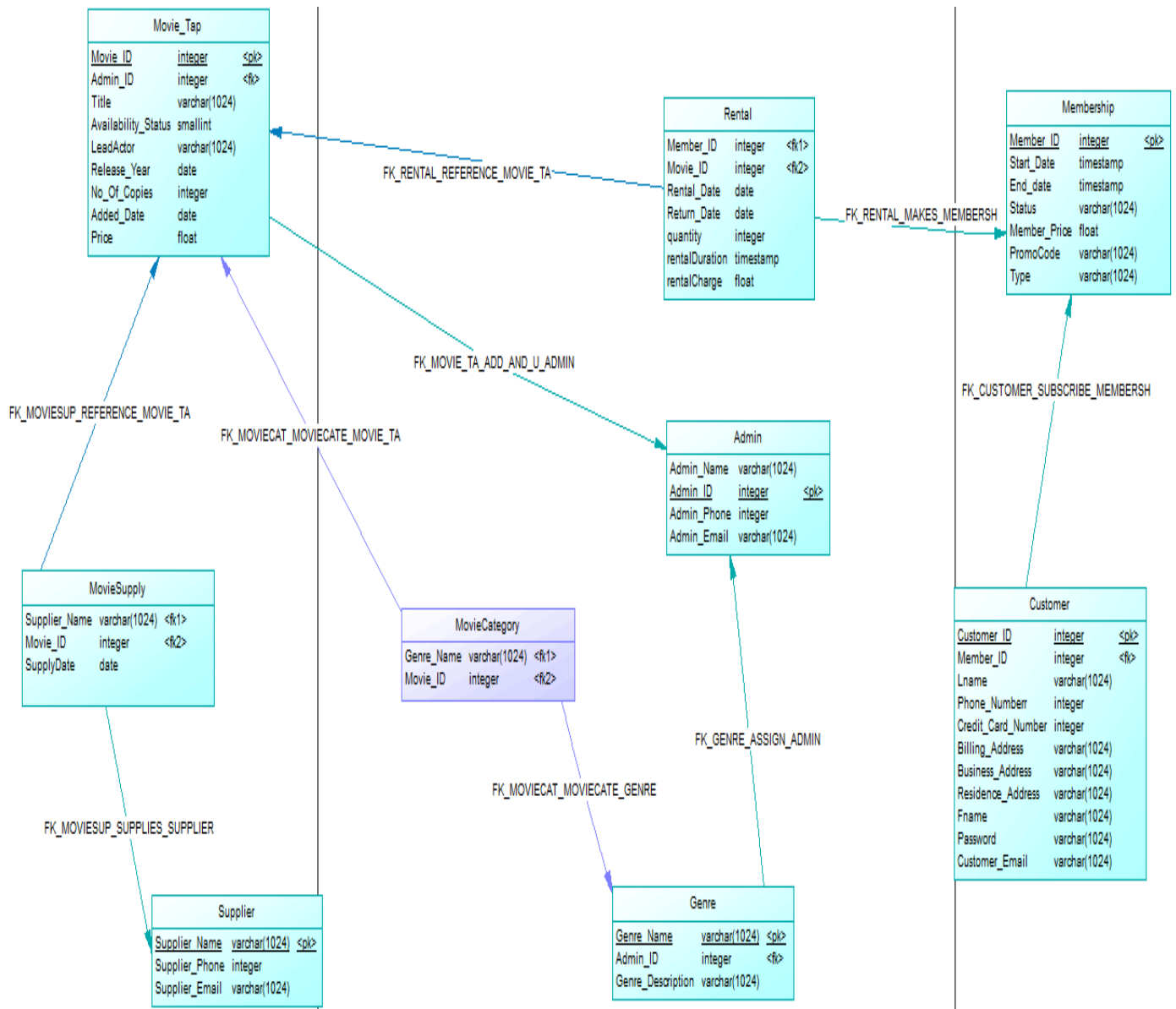
Program: Special Zayed

Phase : 2

1. Conceptual ERD (Power Design)



2.Physical ERD (Power Design)



3.Requirements

1. User Management

Users can:

- Register and maintain personal details (Name, Phone, Addresses (Billing - Residence - Business), Credit Card).
- Update their profile, including addresses and contact details.

2. Movie Management

- Admins can add new movies with metadata (Title, Lead Actor, Release Year, Genre, Availability Status, Number of Copies).
- Admins can categorize movies under multiple genres (via MovieGenre).
- Users can view available movies and search/filter by genre, actor, or availability.
- The system tracks the number of available copies.

3. Membership System

- Users can subscribe to a membership plan(Monthly or Annual).
- The system tracks membership status, start and end dates.
- Only users with an active membership can rent movies.

4. Genre Management

- Admins can Classify movies under one or more genres.
- Movies can belong to multiple genres.

5. Rental Management

- Users can Rent one or more copies of available movies.
- The system calculates rental charges based on duration and quantity.
- Users can Specify duration of rental.
- Users can Return movies (triggering update to availability and return date).
- Rental history is maintained for each user.
- The System can Prevent rentals if insufficient copies are available.

6. Supplier Management

- Admins record movie supplies (date, supplier name, and movie).

1) Business Logic

1. Movie Rental

- A movie can only be rented if AvailabilityStatus = 'Available' and NoOfCopies > 0.
- Rental charges are computed as:
 - $\text{RentalCharge} = \text{Quantity} \times \text{RentalDuration} \times \text{PricePerDay}$.
- When a user rents a movie:
 - NoOfCopies is reduced by the rented quantity.
 - A new record in Rental is created.
- When the user returns the movie:

- NoOfCopies is increased accordingly.
- returnDate is updated in Rental.

2. Membership Requirement

- Rentals can only be made by Members.
- Membership status must be “active” and the current date must be within the StartDate and EndDate.

3. Movie Supply

- A supplier can supply multiple movies.
- Each supply includes a supplyDate.
- When a supply is recorded:
 - NoOfCopies for the movie increases.
- If the movie is not yet in the system, the admin must first add it.

4. Genre & Category

- A movie can belong to multiple genres through the MovieGenre associative entity.
- Each genre is identified by GenreName.

5. Admin Actions (Only users with the Role = 'Admin')

- Add new movies.
- Assign genres to movies.
- Register new supply entries when a supplier gives new stock.
- Admin does not handle rentals or user registration directly.

6. Supply Logic

- When a Supply is recorded:
 - A new MovieSupply entry is created with SupplyDate.
 - NoOfCopies in Movie is updated.

Constraints (Limitations within the Code)

Console-Based Interaction

The original application is designed for command-line interaction, focusing on simplicity and ease of testing via console inputs.

Embedded SQL Queries

All SQL operations are embedded directly within the C# application, allowing low-level control of data queries and transactions.

Straightforward Error Handling

Try-catch blocks are used to catch runtime exceptions and display appropriate error messages, aiding user feedback and stability.

Basic Input Validation

The system performs minimal input checks, such as ensuring numeric input where expected and verifying basic formats (e.g., dates).

Integrated Database Access

The application directly manages database connections and commands without any data access layer or ORM (like Entity Framework).

Direct Password Storage

Passwords are stored in plain text for both Admin and Customer

users. This design simplifies implementation but lacks security practices.

Independent Operation Execution

Each feature (e.g., register, rent, return, view) is handled separately, making the flow modular and easy to follow.

Platform-Specific Configuration

The app is tailored for Microsoft SQL Server with platform-specific connection strings and SQL syntax.

Single-User Session Flow

Each execution session handles one logged-in user at a time, preserving authentication state until the app is closed.

Auto-Generated IDs

Primary key values (e.g., MovieID, MemberID, RentalID) are auto-generated by the database to ensure uniqueness and consistency.

Distinct User Roles

The system supports both Admins and Customers, with separate login flows and functionalities tailored to each.

Application-Level Deletion Logic

Business rules such as disallowing deletion of rented movies are enforced via the application code rather than database constraints.

Graphical User Interface (GUI)

The GUI enhances usability, allowing both Admins and Customers to perform operations like movie rental, return, and reporting.

Multi-Window Workflow Support

Contrary to initial constraints, the GUI supports multiple

forms/windows, enabling structured workflows across tasks like login, rental, and reports.

Assumptions

1. Database Schema Exists

The SQL Server database (**MovieRentalSystem**) and all required tables (e.g., Admin, Customer, Movie_Tape, Rental) are assumed to be pre-created and properly structured.

2. Valid Pre-Existing Data

The database contains valid records that conform to defined constraints (data types, foreign keys, etc.).

3. Auto-Incremented IDs

All primary keys (e.g., MovieID, MemberID) are auto-generated by the database—manual entry is not required or supported.

4. Valid Input Format

Inputs like dates (**yyyy-MM-dd**), decimals, and integers are expected in proper formats by the application.

5. SQL Server Connectivity

The system assumes a stable connection to SQL Server with the appropriate credentials and access permissions.

6. Required Permissions

The app assumes that the executing user has sufficient privileges for SQL operations (SELECT, INSERT, UPDATE, DELETE).

7. No External Libraries

The solution uses only the .NET base libraries and **Microsoft.Data.SqlClient**. No third-party packages or services are required.

Additional Assumptions (GUI & Report)

1. Windows Environment: GUI requires a Windows OS with .NET Desktop Runtime installed.
2. Test Data Availability: Report queries expect the database to be populated with meaningful data.
3. Basic GUI Proficiency: Users are assumed to understand basic GUI interaction (e.g., clicking buttons).
4. Admin Use for Reports: Reports are expected to be generated by an Admin user.

Project Evolution

This project was initially built as a console-based system for educational purposes. To enhance functionality and usability:

- A GUI using WinForms was added for simplified operation.
- A reporting module was implemented using SQL queries displayed in **DataGridView** controls.

